Advanced Binary Search Tree Algorithms: Kth Smallest, Morris Traversal, Lowest Common Ancestor, and BST Recovery — A Formal Unified Analysis

Abstract

This paper presents a formal, unified algorithmic analysis of four advanced Binary Search Tree (BST) problems: (1) finding the k-th smallest element, (2) Morris Traversal for $O(1)$-space inorder traversal via threaded binary tree links, (3) computing the Lowest Common Ancestor (LCA) in both general Binary Trees and BSTs, and (4) recovering a BST from exactly two swapped nodes. We prove that all four problems reduce to controlled instances of BST inorder traversal, exploiting the BST Inorder Invariant as a unifying primitive. Complete algorithm correctness proofs, complexity analyses, and comparative tables are provided. Key results include: a formal proof that Morris Traversal achieves $O(n)$ time and $O(1)$ auxiliary space while fully restoring tree structure; a proof that BST-LCA runs in $O(\log n)$ time for balanced trees; and an inversion-counting argument for $O(n)$ BST recovery, with an $O(1)$-space variant combining Morris Traversal with inversion detection. These results have direct applications in database B-tree indexing, compiler AST optimization, and space-constrained embedded system algorithms.

Index Terms

binary search trees, Morris traversal, lowest common ancestor, BST recovery, inorder traversal, algorithmic complexity, threaded binary trees, tree algorithms

1 Introduction

Binary Search Trees (BSTs) are foundational data structures in computer science, enabling ordered storage with average-case $O(\log n)$ search, insertion, and deletion operations. Despite their ubiquity in database indexing, compiler symbol tables, and priority queuing systems, several advanced algorithmic challenges remain pedagogically significant and practically relevant. This paper formalizes four such advanced BST and Binary Tree problems drawn from first-principles analysis: (1) finding the k-th smallest element in a BST, (2) Morris Traversal for space-optimal inorder traversal, (3) finding the Lowest Common Ancestor (LCA) in a Binary Tree and BST, and (4) recovering a BST from exactly two swapped nodes.

We present complete algorithmic solutions, formal proofs of correctness, complexity analysis, and illustrative code implementations. The novelty of this treatment lies in the unified formal framework connecting all four problems via inorder traversal, threading, and BST structural invariants.

1.1 Background and Motivation

A Binary Search Tree T is defined recursively: for every node $v \in T$, all values in the left subtree of v are strictly less than $v.\text{val}$, and all values in the right subtree of v are strictly greater than $v.\text{val}$. This invariant implies a critical structural property:

Theorem 1 (BST Inorder Theorem). The inorder traversal of a valid BST T produces a strictly monotonically increasing sequence. Formally, let $\sigma = \langle v_1, v_2, \ldots, v_n \rangle$ be the inorder traversal sequence of T. Then:

$$v_1.\text{val} < v_2.\text{val} < \cdots < v_n.\text{val}$$

This theorem serves as the foundational invariant exploited in all four algorithmic problems analyzed in this paper.

2 Kth Smallest Element in a BST

2.1 Problem Statement

Given a Binary Search Tree T with n nodes and an integer k such that $1 \leq k \leq n$, find the value of the k-th smallest element. This problem has practical applications in database range queries, statistical rank computation, and order-statistic trees.

2.2 Approach I: Inorder Traversal with Array

By the BST Inorder Theorem, the k-th element in the inorder traversal corresponds exactly to the k-th smallest element.

```
void inorderHelper(TreeNode root, List<Integer> res) {
    if (root == null) return;
    inorderHelper(root.left, res);
    res.add(root.val);
    inorderHelper(root.right, res);
}

int kthSmallest(TreeNode root, int k) {
    List<Integer> res = new ArrayList<>();
    inorderHelper(root, res);
    return res.get(k - 1);
}
```

Complexity: Time $O(n)$, Space $O(n)$ for the auxiliary list.

2.3 Approach II: Recursive Inorder with Global Counter

To avoid storing all n values, we maintain a global counter c that increments at each inorder visit and terminates early when $c = k$.

```
int count = 0, result = -1;

void inorder(TreeNode root, int k) {
    if (root == null) return;
    inorder(root.left, k);
    count++;
    if (count == k) { result = root.val; return; }
    inorder(root.right, k);
}
```

Complexity: Time $O(k)$ best case, $O(n)$ worst case. Space $O(h)$ for the call stack.

Theorem 2 (Correctness of k-th Smallest). The recursive inorder traversal visits nodes in strictly increasing order. The counter c increments in exact correspondence with inorder visits. When $c = k$, the current node is the k-th smallest.

Proof. By the BST Inorder Theorem, inorder sequence $\sigma = \langle v_1, v_2, \ldots, v_n \rangle$ is strictly sorted. Counter c starts at 0 and reaches k precisely when the k-th inorder node v_k is visited. By the monotonicity of σ, v_k is the k-th smallest value. □

3 Morris Traversal: Space-Optimal Inorder Traversal

3.1 Motivation

Standard inorder traversal requires $O(h)$ auxiliary space for the call stack, where $h = O(n)$ for degenerate trees. Morris Traversal [1] achieves $O(n)$ time and $O(1)$ space by temporarily modifying tree structure using threaded links.

Definition 3 (Inorder Predecessor). For a node v with a non-null left child, the inorder predecessor is:

$$\text{pred}(v) = \text{rightmost}(v.\text{left})$$

3.2 Algorithm

```
List<Integer> morrisInorder(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    TreeNode curr = root;

    while (curr != null) {
        if (curr.left == null) {
            result.add(curr.val);
            curr = curr.right;
        } else {
            TreeNode pred = curr.left;
            while (pred.right != null && pred.right != curr)
                pred = pred.right;
```

```
            if (pred.right == null) {
                pred.right = curr;   // Create thread
                curr = curr.left;
            } else {
                pred.right = null;   // Break thread
                result.add(curr.val);
                curr = curr.right;
            }
        }
    }
    return result;
}
```

3.3 Correctness and Complexity

Theorem 4 (Morris Traversal Correctness). Morris Traversal visits all nodes exactly once in inorder sequence and fully restores the original tree structure.

Proof. By structural induction on n.

Base case ($n = 1$): Single node has no left child. Case 1 applies: visit, move right, terminate. Structure unchanged. ✓

Inductive step: For root r with left subtree L and right subtree R:

1) Predecessor $p = \text{pred}(r)$ is found in L. Thread $p.right \leftarrow r$ created.
2) Traversal descends into L. By induction, all nodes of L are visited in inorder.
3) When p is revisited: $p.right = r \neq \text{null}$ (thread detected). Thread broken: $p.right \leftarrow \text{null}$. Node r visited.
4) Traversal proceeds to R. By induction, all nodes of R visited in inorder.

Visit order is L (inorder) $\rightarrow r \rightarrow R$ (inorder) = correct inorder for T. Thread created and destroyed symmetrically, restoring structure. □

Theorem 5 (Morris Traversal Complexity). Time: $O(n)$. Space: $O(1)$.

Proof. Space: Only constant pointers curr, pred. No stack or recursion. $O(1)$.

Time: Each node visited ≤ 2 times (thread creation and thread breaking). Each right-pointer in left subtrees traversed ≤ 2 times total across all iterations. Total pointer operations = $O(n)$. □

TABLE 1
Comparison of Inorder Traversal Approaches

Method	Time	Space	Modifies Tree?
Recursive	$O(n)$	$O(h)$	No
Iterative (Stack)	$O(n)$	$O(h)$	No
Morris Traversal	$O(n)$	$O(1)$	Temporarily

4 Lowest Common Ancestor

4.1 Problem Statement

Given a Binary Tree T and two nodes A, B in T, the Lowest Common Ancestor (LCA) is the deepest node v such that v is an ancestor of both A and B. A node is its own ancestor.

4.2 LCA in General Binary Trees

```
TreeNode lca(TreeNode root, int A, int B) {
    if (root == null) return null;
    if (root.val == A || root.val == B) return root;

    TreeNode left  = lca(root.left,  A, B);
    TreeNode right = lca(root.right, A, B);

    if (left != null && right != null) return root;
    return (left != null) ? left : right;
}
```

Theorem 6 (LCA Correctness — Guaranteed Presence). If A and B are both present in T, the above algorithm returns the correct LCA in $O(n)$ time, $O(h)$ space.

Proof. Let v^* be the true LCA.

Case 1: $v^* = A$ or $v^* = B$. Base case returns v^* immediately. Correct, since a node is its own ancestor.

Case 2: A and B are in different subtrees of v^*. Recursive calls return non-null from both children of v^*. Both left and right are non-null at v^*, which is returned. Every node is visited once, giving $O(n)$ time. □

When existence is not guaranteed, verify presence first:

```
boolean find(TreeNode root, int target) {
   if (root == null) return false;
   return root.val == target ||
         find(root.left, target) ||
         find(root.right, target);
}

TreeNode solve(TreeNode root, int A, int B) {
   if (!find(root, A) || !find(root, B)) return null;
   return lca(root, A, B);
}
```

4.3 LCA in BSTs: Exploiting Ordering

For a BST with nodes $A \leq B$, the LCA is the first node where $A \leq v.\text{val} \leq B$:

```
TreeNode lcaBST(TreeNode root, int A, int B) {
   if (root == null || root.val == A || root.val == B)
      return root;
   if (root.val > B) return lcaBST(root.left,  A, B);
   if (root.val < A) return lcaBST(root.right, A, B);
   return root;
}
```

Theorem 7 (BST LCA Complexity). For a balanced BST of height $h = O(\log n)$, LCA runs in $O(\log n)$ time and $O(1)$ iterative space.

Proof. At each node, one comparison directs traversal to exactly one subtree. Recursion depth $\leq h$. For balanced BST, $h = O(\log n)$. Implemented iteratively, space is $O(1)$. □

TABLE 2
LCA Complexity Comparison

Method	Time	Space
Binary Tree	$O(n)$	$O(h)$
BST (balanced)	$O(\log n)$	$O(1)$ iterative
BST (degenerate)	$O(n)$	$O(1)$ iterative

5 Recover BST: Two Swapped Nodes

5.1 Problem Statement

Given a BST where exactly two nodes have been swapped, restore the BST without altering its structure.

5.2 Core Observation

By the BST Inorder Theorem, swapping two nodes v_i and v_j $(i < j)$ in the inorder sequence produces exactly 1 or 2 consecutive inversions:

- Adjacent ($j = i + 1$): One inversion at position i.
- Non-adjacent ($j > i + 1$): Two inversions, at positions i and $j - 1$.

5.3 Algorithm: O(n) Inversion Detection

```
TreeNode first = null, second = null, prev = null;

void detect(TreeNode root) {
    if (root == null) return;
    detect(root.left);
    if (prev != null && prev.val > root.val) {
        if (first == null) first = prev;
        second = root;
    }
    prev = root;
    detect(root.right);
}

void recoverTree(TreeNode root) {
    detect(root);
    int tmp = first.val;
    first.val = second.val;
    second.val = tmp;
}
```

Theorem 8 (Correctness of BST Recovery). Swapping first.val and second.val after inversion detection restores the BST.

Proof. Let $\sigma^* = \langle a_1, \ldots, a_n \rangle$ and σ be the corrupted sequence with a_i, a_j swapped.

Adjacent case: One inversion at i. Algorithm sets first $= v_i$ (value a_j), second $= v_{i+1}$ (value a_i). Swap restores σ^*.

Non-adjacent case: First inversion at i: first $= v_i$ (value a_j). Second inversion at $j-1$: second $= v_j$ (value a_i). Swap restores σ^*. □ □

5.4 Morris-Enhanced O(1) Space Recovery

By integrating inversion detection into Morris Traversal, we achieve BST recovery in $O(n)$ time and $O(1)$ space:

```
void recoverTreeMorris(TreeNode root) {
    TreeNode first=null, second=null, prev=null, curr=root;
    while (curr != null) {
        if (curr.left == null) {
            if (prev != null && prev.val > curr.val) {
                if (first == null) first = prev;
                second = curr;
            }
            prev = curr; curr = curr.right;
        } else {
            TreeNode pred = curr.left;
            while (pred.right != null && pred.right != curr)
                pred = pred.right;
            if (pred.right == null) {
                pred.right = curr; curr = curr.left;
            } else {
                pred.right = null;
                if (prev != null && prev.val > curr.val) {
                    if (first == null) first = prev;
                    second = curr;
                }
                prev = curr; curr = curr.right;
            }
        }
    }
    int tmp = first.val; first.val = second.val; second.val = tmp;
}
```

TABLE 3
Recover BST Complexity Comparison

Method	Time	Space	Notes
Sort-based	$O(n \log n)$	$O(n)$	Simple
Inorder + Inversion	$O(n)$	$O(h)$	Efficient
Morris + Inversion	$O(n)$	$O(1)$	Optimal

6 Unified Framework

Corollary 9 (Inorder as Universal BST Primitive). All four algorithms reduce to controlled inorder traversal of BSTs:

- k-th Smallest: Terminate at inorder visit k.
- Morris Traversal: Perform inorder in $O(1)$ space via predecessor threading.
- LCA (BST): Navigate to first node v with $A \leq v.\text{val} \leq B$.
- Recover BST: Detect inversions during inorder to find swapped nodes.

This unification motivates a key pedagogical principle: mastery of BST inorder traversal—including its space-time tradeoffs across recursive, iterative, and Morris variants—unlocks efficient solutions to a wide class of advanced tree problems.

7 Conclusion

This paper presented a formal, unified treatment of four advanced BST algorithms connected by the BST Inorder Invariant. Key contributions include:

- A formal inductive proof that Morris Traversal correctly visits all nodes in inorder order in $O(n)$ time and $O(1)$ space while fully restoring tree structure.
- A proof that BST-LCA achieves $O(\log n)$ time for balanced trees by exploiting BST ordering to halve the search space at each level.
- A rigorous inversion-counting argument proving the correctness of $O(n)$ BST recovery, with an $O(1)$-space variant via Morris integration.
- Comparative complexity tables enabling principled algorithm selection for space-constrained deployments.

Future work includes extending Morris Traversal to generalized k-ary trees, designing cache-oblivious BST traversal algorithms, and quantifying practical performance impacts of $O(1)$-space traversal in large-scale in-memory database systems.

References

[1] J. H. Morris Jr., "Traversing binary trees simply and cheaply," Information Processing Letters, vol. 9, no. 5, pp. 197–200, 1979.
[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. MIT Press, 2009.
[3] D. E. Knuth, The Art of Computer Programming, Vol. 1: Fundamental Algorithms, 3rd ed. Addison-Wesley, 1997.
[4] S. S. Skiena, The Algorithm Design Manual, 2nd ed. Springer, 2008.